

Bases de données et langage SQL

SAÉ S104 Création d'une base de données

Rapportage de la saison balnéaire 2024

Nom : THIAM

Prénoms : Mouhamed Fadel

Promotion : BUT 1 Informatique

Groupe : Cérés

Sommaire

Introduction	2
Script SQL de création des tables	2
Illustrations comparatives AGL/Cours commentées d'une association fonctionnelle	4
Illustrations comparatives AGL/Cours commentées d'une association maillée	4
Modèle physique de données réalisé avec l'AGL	5
Script SQL de création des tables généré automatiquement par l'AGL	6
Discussion sur les différences entre les scripts produits manuellement et automatiquement	7
Script de peuplement de la base de données	8
Description commentée des différentes étapes du script de peuplement	10

Introduction

Cette SAÉ consiste à créer une base de données où on aura les données relatives à la saison balnéaire 2024 notamment les mesures bactériologiques d'entérocoques intestinaux et d'Escherichia coli sur des prélèvements d'eau des lieux de baignade en France et aussi des événements survenus au cours de cette saison.

Pour ce faire, le SGBD Postgresql a été imposé et le choix de l'AGL est libre.

Pour répondre aux demandes de notre SAÉ, nous avons donc utilisé l'AGL **DB Designer**.

Script SQL de création des tables

DROP TABLE IF EXISTS événements;

DROP TABLE IF EXISTS analyses;

DROP TABLE IF EXISTS sites;

DROP TABLE IF EXISTS communes;

DROP TABLE IF EXISTS départements;

DROP TABLE IF EXISTS régions;

CREATE TABLE régions

(
 code_region **VARCHAR** PRIMARY KEY,
 nom_region **VARCHAR**

);

CREATE TABLE départements

(
 code_departement **VARCHAR** PRIMARY KEY,
 region **VARCHAR**,
 nom_departement **VARCHAR**,
 CONSTRAINT fk_region FOREIGN KEY (region) REFERENCES
régions(code_region) **ON DELETE** CASCADE
);

CREATE TABLE communes

(
 code_commune **VARCHAR** PRIMARY KEY,
 departement **VARCHAR**,
 nom_commune **VARCHAR**,
 CONSTRAINT fk_departement FOREIGN KEY (departement) REFERENCES
départements(code_departement) **ON DELETE** CASCADE
);

CREATE TABLE sites

(
 idSite **VARCHAR** PRIMARY KEY,
 codeCommune **VARCHAR**,
 nom_site **VARCHAR**,
 dateDeclaration **DATE**,

```

        TypeEau VARCHAR,
        longitude VARCHAR ,
        latitude VARCHAR ,
        CONSTRAINT fk_codeCommune FOREIGN KEY (codeCommune)
REFERENCES communes(code_commune) ON DELETE CASCADE
);

```

CREATE TABLE événements

```

(
    idEvenement INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    idSite VARCHAR,
    evenement VARCHAR,
    debut DATE,
    fin DATE,
    mesure VARCHAR,
    CONSTRAINT fk_idSite FOREIGN KEY (idSite) REFERENCES sites(idSite) ON
DELETE CASCADE
);

```

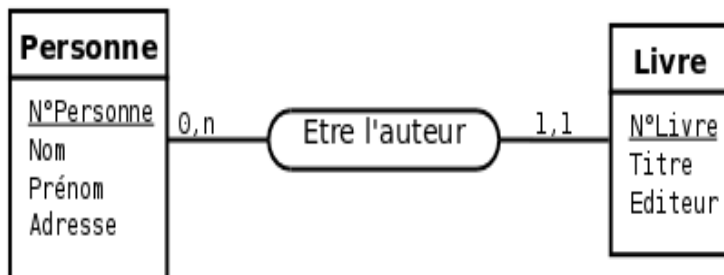
CREATE TABLE analyses

```

(
    idAnalyse INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    idSite VARCHAR,
    datePrelevement DATE,
    enterocoques INTEGER,
    escherichia INTEGER,
    CONSTRAINT fk_idSite FOREIGN KEY (idSite) REFERENCES sites(idSite) ON
DELETE CASCADE
);

```

Illustrations comparatives AGL/Cours commentées d'une association fonctionnelle

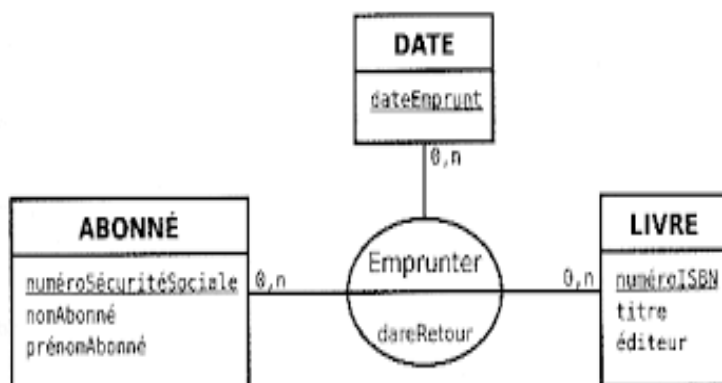


personne	
N°personne	integer
N°livre	integer
Nom	varchar
Prenom	varchar
Adresse	varchar

livres	
N°livre	integer
Titre	varchar
Editeur	varchar

L'association fonctionnelle vue en cours avec les cardinalités dont au moins une des cardinalités est 1 est représentée avec une flèche qui pointe vers la table ayant la cardinalité 1. Ceci est dû au fait que chaque élément de la table Personne dans notre exemple pointe vers un unique élément dans Livre car dans l'exemple, un livre n'est écrit que par une seule personne alors qu'une personne peut écrire plusieurs livres.

Illustrations comparatives AGL/Cours commentées d'une association maillée

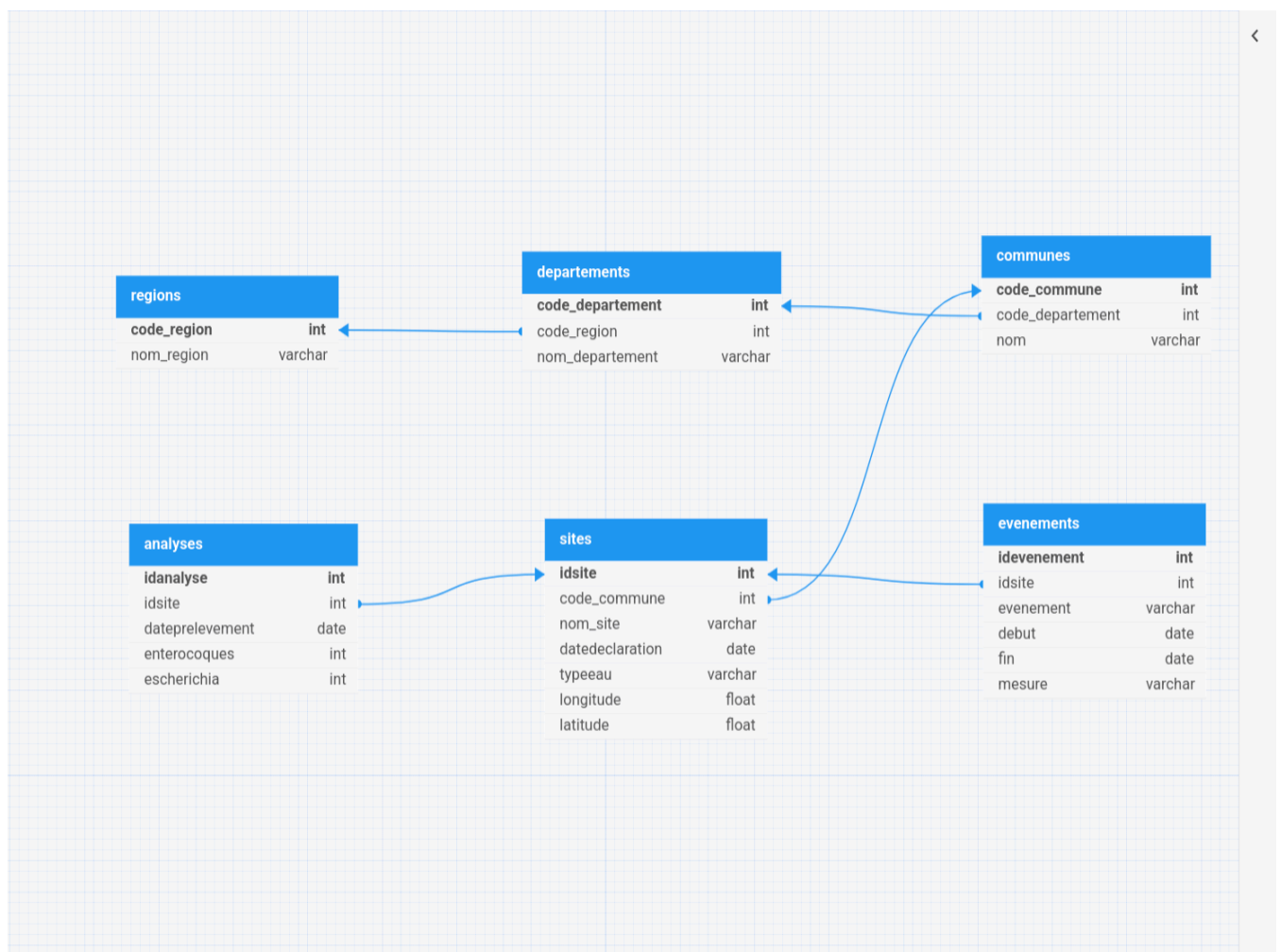


livres	
numero_ISBN	bigint
titre	varchar
editeur	varchar

abonnes	
numero_secu	bigint
numero_ISBN	bigint
nom	varchar
prenom	varchar

L'association maillée vue en cours avec les cardinalités n,n, où chaque table inclue dans l'association a sa cardinalité (n dans le cas d'une maillée) qui est précisée devant elle, est représentée par l'AGL avec des flèches en forme de pattes d'oie qui illustre qu'un élément peut être lié à plusieurs autres éléments de la table associée. Dans notre exemple, cela montre qu'un abonné peut emprunter plusieurs livres à différentes dates et inversement un livre peut être emprunté par plusieurs personnes à des dates différentes.

Modèle physique de données réalisé avec l'AGL



Script SQL de création des tables généré automatiquement par l'AGL

```
CREATE TABLE IF NOT EXISTS "regions" (  
  "code_region" bigint NOT NULL,  
  "nom_region" varchar NOT NULL,  
  PRIMARY KEY ("code_region")  
);
```

```
CREATE TABLE IF NOT EXISTS "departements" (  
  "code_departement" bigint NOT NULL,  
  "code_region" bigint NOT NULL,  
  "nom_departement" varchar NOT NULL,  
  PRIMARY KEY ("code_departement")  
);
```

```
CREATE TABLE IF NOT EXISTS "communes" (  
  "code_commune" bigint NOT NULL,  
  "code_departement" bigint NOT NULL,  
  "nom" varchar NOT NULL,  
  PRIMARY KEY ("code_commune")  
);
```

```
CREATE TABLE IF NOT EXISTS "sites" (  
  "idsite" bigint NOT NULL,  
  "code_commune" bigint NOT NULL,  
  "nom_site" varchar NOT NULL,  
  "datedeclaration" date NOT NULL,  
  "typeeau" varchar NOT NULL,  
  "longitude" double precision NOT NULL,  
  "latitude" double precision NOT NULL,  
  PRIMARY KEY ("idsite")  
);
```

```
CREATE TABLE IF NOT EXISTS "evenements" (  
  "idevenement" bigint NOT NULL,  
  "idsite" bigint NOT NULL,  
  "evenement" varchar NOT NULL,
```

```

"debut" date NOT NULL,
"fin" date NOT NULL,
"mesure" varchar NOT NULL,
PRIMARY KEY ("idevenement")
);

```

```

CREATE TABLE IF NOT EXISTS "analyses" (
  "idanalyse" bigint NOT NULL,
  "idsite" bigint NOT NULL,
  "dateprelevement" date NOT NULL,
  "enterocoques" bigint NOT NULL,
  "escherichia" bigint NOT NULL,
  PRIMARY KEY ("idanalyse")
);

```

```

ALTER TABLE "departements" ADD CONSTRAINT "departements_fk1" FOREIGN KEY
("code_region") REFERENCES "regions"("code_region");
ALTER TABLE "communes" ADD CONSTRAINT "communes_fk1" FOREIGN KEY
("code_departement") REFERENCES "departements"("code_departement");
ALTER TABLE "sites" ADD CONSTRAINT "sites_fk1" FOREIGN KEY
("code_commune") REFERENCES "communes"("code_commune");
ALTER TABLE "evenements" ADD CONSTRAINT "evenements_fk1" FOREIGN KEY
("idsite") REFERENCES "sites"("idsite");
ALTER TABLE "analyses" ADD CONSTRAINT "analyses_fk1" FOREIGN KEY ("idsite")
REFERENCES "sites"("idsite");

```

Discussion sur les différences entre les scripts produits manuellement et automatiquement

En observant les deux scripts celui réalisé par l'AGL et le nôtre, nous constatons que l'AGL crée d'abord ses tables avec tous ses attributs et à la fin il rajoute des contraintes avec la requête "ALTER TABLE" qui permet de modifier le contenu et les paramètres de la table comme pour ajouter des clés étrangères. De même pour montrer les clés primaires il préfère les identifier en toute fin de sa requête "CREATE TABLE". Ceci est pour éviter les ambiguïtés et davantage faciliter le travail pour le cas des clés combinées.

On note que par rapport à notre script, celui de l'AGL est plus affiné pour éviter toute imprécision. C'est dans ce cas que le script de l'AGL utilise le type "bigint" au détriment du type "integer" pour préciser que c'est des grands nombres et aussi "double precision" au lieu de float qui ne serait pas très adapté dans notre cas de figure.

Script de peuplement de la base de données

--SUPPRESSION_DE_TABLES

```
DROP TABLE IF EXISTS ANALYSES_temp;  
DROP TABLE IF EXISTS DEPARTEMENTS;  
DROP TABLE IF EXISTS INFORMATIONS;  
DROP TABLE IF EXISTS SITES_B;
```

-- CREATION_DE_TABLES_TEMPORAIRES

CREATE TABLE SITES_B

```
(  
    Saison_balneaire INTEGER,  
    Region VARCHAR,  
    Département VARCHAR,  
    Code_unique_d_identification_du_site_de_baignade VARCHAR,  
    Precedent_code_unique_d_identification_du_site_de_baignade VARCHAR,  
    Evolution_2024_vs_2023 VARCHAR,  
    Nom_du_site_de_baignade VARCHAR,  
    Code_INSEE_de_la_commune VARCHAR,  
    Nom_de_la_commune VARCHAR,  
    Date_declaration_UE DATE,  
    Type_d_eau VARCHAR,  
    Longitude VARCHAR,  
    Latitude VARCHAR  
);
```

CREATE TABLE INFORMATIONS

```
(  
    Saison_balneaire INTEGER,  
    Region VARCHAR,  
    Departement VARCHAR,  
    Code_unique_d_identification_du_site_de_baignade VARCHAR,  
    Type_d_evenement VARCHAR,  
    Date_de_debut DATE,  
    Date_de_fin DATE,  
    Mesures_de_gestion VARCHAR  
);
```

CREATE TABLE DEPARTEMENTS

```
(  
    code_departement VARCHAR,  
    nom_departement VARCHAR,  
    code_region INTEGER,  
    nom_region VARCHAR  
);
```



```

CREATE TABLE ANALYSES_temp
(
    Saison_balneaire INTEGER,
    Region VARCHAR,
    Departement VARCHAR,
    Code_unique_d_identification_du_site_de_baignade VARCHAR,
    Date_de_prelevement DATE,
    Resultat_enterocoques_intestinaux INTEGER,
    Resultat_Escherichia_coli INTEGER,
    Statut_du_prelevement VARCHAR,
    standard VARCHAR,
    Standard2 VARCHAR,
    standard3 VARCHAR
);

-- REMPLISSAGE_DE_TABLES_TEMPORAIRES

\copy SITES_B FROM liste-des-sites-de-baignade-saison-balneaire-2024.csv
WITH (
    FORMAT csv,
    HEADER true,
    DELIMITER ';',
    ENCODING 'LATIN1'
)
\copy INFORMATIONS FROM saison-balneaire-2024-informations-sur-la-saison.csv
WITH (
    FORMAT csv,
    HEADER true,
    DELIMITER ';',
    ENCODING 'LATIN1'
)
\copy ANALYSES_temp FROM saison-balneaire-2024-resultats-danalyses.csv
WITH (
    FORMAT csv,
    HEADER true,
    DELIMITER ';',
    ENCODING 'LATIN1'
)
\copy DEPARTEMENTS FROM departements-france.csv
WITH (
    FORMAT csv,
    HEADER true,
    DELIMITER ';'
)

-- REMPLISSAGE_BD

```

```

INSERT INTO régions(code_region,nom_region)
SELECT DISTINCT code_region, nom_region
FROM DEPARTEMENTS JOIN régions USING (code_region);
INSERT INTO départements(code_departement,region,nom_departement)
SELECT DISTINCT LTRIM(code_departement,'0'), code_region, nom_departement
FROM DEPARTEMENTS JOIN départements USING (code_departement) ;
INSERT INTO communes(code_commune, departement, nom_commune)
SELECT DISTINCT code_insee_de_la_commune, LTRIM(département,'0'),
nom_de_la_commune FROM SITES_B JOIN communes ON
code_insee_de_la_commune=code_commune;
INSERT INTO sites(idSite, nom_site, codeCommune, dateDeclaration, typeEau,
longitude, latitude)
SELECT DISTINCT code_unique_d_identification_du_site_de_baignade,
nom_du_site_de_baignade, code_insee_de_la_commune, date_declaration_UE,
type_d_eau, longitude, latitude FROM SITES_B JOIN sites ON
code_unique_d_identification_du_site_de_baignade = idSite ;
INSERT INTO événements(idSite, evenement, debut, fin, mesure)
SELECT DISTINCT code_unique_d_identification_du_site_de_baignade,
Type_d_evenement, Date_de_debut, Date_de_fin, Mesures_de_gestion FROM
INFORMATIONS;
INSERT INTO analyses(idSite, datePrelevement, enterocoques, escherichia)
SELECT DISTINCT code_unique_d_identification_du_site_de_baignade,
date_de_prelevement, resultat_enterocoques_intestinaux, resultat_escherichia_coli
FROM ANALYSES_temp;

```

Description commentée des différentes étapes du script de peuplement

Pour remplir nos tables via les fichiers csv téléchargés, nous avons eu recours à la création de tables temporaires identiques (même attributs) aux tables contenues dans les fichiers csv. Nous avons ensuite utilisé la commande \copy pour remplir les tables temporaires. Une fois les tables remplies, il ne nous restait qu'à utiliser la requête "INSERT INTO ... SELECT ... FROM ..." pour peupler nos tables en respectant toutes les contraintes. Ainsi pour idAnalyse et idEvenement, nous avons eu à utiliser "GENERATED ALWAYS AS IDENTITY" lors de la création des tables parce qu'il n'ont pas été référencé dans les tables comprises dans les fichiers. Il s'avère que les codes des départements dans les fichiers étaient précédés de 0 donc nous avons eu à utiliser LTRIM(...,'0') pour supprimer ces 0. Les données des tables départements et régions ont été tirées de la table temporaire DEPARTEMENTS, les tables communes et sites ont été remplies grâce à la table SITES_B, la table analyses a été remplie avec ANALYSES_temp et événements par le biais de la table INFORMATIONS.